

TRANSFORMER BASED SYMBOLIC MUSIC GENERATION

Wongee Hong

The Cooper Union
Albert Nerken School of Engineering

Lennart Vetter

The Cooper Union
Albert Nerken School of Engineering

ABSTRACT

This work addresses the limitations of recurrent architectures and small-scale datasets in symbolic music generation. Building upon previous attempts based on LSTMs, which yielded suboptimal results in musical coherence, we propose a transition to a transformer-based architecture. By aggregating three distinct datasets (VGMIDI, EMOPIA, and Pop1K7), we train a decoder-style transformer on a large-scale corpus of approximately 2.25 million tokens. Our model employs a vocabulary of 1,350 unique tokens to represent melody and harmony. Preliminary results indicate that while the model learns basic structural dependencies, as evidenced by n-gram structural cosine similarity metrics, the transition from LSTM to Transformer significantly improves the stability of the generated musical sequences while also pointing out the need for large scale datasets to satisfy the intensive data demand of these models. This demonstrates the viability of employing transformer architectures for music generation in the current ubiquitous Big Data regime.

1. INTRODUCTION

Symbolic music generation has evolved from rule-based systems [1] to sophisticated deep generative models capable of capturing complex and nuanced polyphonic structures [2]. However, two primary challenges persist: 1. the effective modeling of long-range dependencies in full-song compositions and 2. the effective training on large scale datasets. Previous work [3] attempted to use Long Short-Term Memory (LSTM) networks trained on the VGMIDI dataset. However, these autoregressive models like LSTM and RNNs often suffered from "drift," where the lack of long-range attention leads to a loss of structural integrity due to vanishing gradient signals during back propagation in time for long sequences, thus yielding music pieces with incoherent musical structure.

In this project, we test the hypothesis that music generation is essentially a data-hungry task that requires high-capacity architectures, which are capable of efficiently scaling to large datasets while capturing long ranging global dependencies present in music. We pivot from LSTMs to a transformer architecture and expand our training data by a factor of 10, incorporating the EMOPIA and Pop1K7 datasets. We focus on the "quality-first" approach: establishing a baseline of high-quality music generation first and leaving the integration of sentiment steering up to further research.

2. RELATED WORK

The field of Symbolic Music Generation has increasingly pivoted towards high-capacity generative models, necessitated by the complexity of capturing polyphonic dependencies and efficient training. This section contextualizes our work within the evolving landscape of token-based music generation.

Early efforts in affective algorithmic composition primarily relied on rule-based systems to map musical features in either categorical or dimensional spaces [4]. For instance, systems like TransPose employ rule-based piano melody generation from text [1]. Similarly, MetaCompose employed pre-defined chord progression graphs to generate real-time game soundtracks [5]. The adoption of Deep Learning has shifted the focus toward learning complex polyphonic structures directly from data. However, achieving structurally coherent and human like music remains a primary challenge. Ferreira and Whitehead [3] utilized a single-layer multiplicative Long Short-Term Memory (LSTM) network with 4,096 units. This model was trained as a generative language model to predict the next musical "word" in a custom 225-token vocabulary representing pitch, duration, velocity, and tempo. However, they noted that the produced music often exhibited structural inconsistency. This shows that while in theory recurrent models like LSTM are able to model long range dependencies, these models struggle to maintain coherence in practice.

Inspired by recent advances in NLP, transformer-based models seem to be excellent at long range sequence modeling. This observation serves as the primary catalyst for the transition to the transformer-based architecture proposed in this work. Recent surveys support this intuition [6] highlighting the shift from RNNs to self-attention mechanisms and noting that transformers better capture the periodic and hierarchical nature of music.

Transformer architectures are considered data hungry, which leads us to the question of scale and quality of the training data. [7] demonstrated that pre-training on a large, unlabeled data corpus with subsequent finetuning significantly improves the subjective "humanness" and "richness" of generated pieces. We extend this paradigm by aggregating three distinct piano datasets VGMIDI [3], EMOPIA [7], and Pop1K7 [2] to maximize the model's exposure to diverse harmonic and emotional structures. Compared to modern large-scale corpora used in NLP, the proposed dataset is still considered small in scale but marks a step in the direction of Big Data regimes in Symbolic Music

Generation.

The effectiveness of transformer-based music generation is heavily predicated on the choice of symbolic encoding. Several encoding schemes have been proposed. A Midi like encoding has been proposed by [8] which uses `note-on`, `note-off`, and `velocity` tokens, alongside `time-shift` tokens to represent absolute time intervals in milliseconds. Introduced by [9], the REMI style encoding adopts a metrical, beat-based approach using `bar` and `sub beat` tokens. It replaces `note-off` with explicit `duration` tokens and includes `tempo` tokens, providing a more rhythmically structured representation that aligns better with musical theory. More recent approaches use compound word tokenization which further optimizes sequence length and computational efficiency [2]. In this scheme, related tokens (e.g., pitch, duration, and velocity) are grouped into a single "super token" or compound word at each time step which reduces the total vocabulary size thus enabling the transformer to attend to longer sequences, yielding better musical structure and model training properties.

Our work makes the following contributions to the research community:

- Proposes a large pop piano dataset for symbolic music generation
- Trains a Decoder-style transformer
- Outlines the necessity to move to a Big Data Regime for Music Generation

3. EXPERIMENTAL SETUP

The following sections will outline the approach taken to train our transformer architecture as well as the compilation of the dataset we trained on. We treat music modeling as a language modeling task. Musical events are encoded according to the encoding scheme proposed in [3] utilizing `pitch`, `duration`, `velocity` and `tempo` tokens. We chose this approach for its more simple design when compared to more complex tokenization schemes. This yields a vocabulary of $n = 1,350$ distinct tokens.

3.1 Data

We combine three commonly used piano datasets in order to increase the training data size. The EMOPIA dataset [7] provides a multi-modal (audio/MIDI) foundation for emotion-labeled pop piano. It is composed of 1087 music clips from 387 songs transcribed from YouTube, encompassing a diverse set of playing styles and high musical quality. This dataset contributes 480,335 tokens to the final data mix.

The Pop1K7 dataset [2] is composed of machine transcriptions of 1,747 audio recordings of piano covers of Japanese anime, Korean and Western pop music, amounting to over 100 hours worth of data. The transcription was

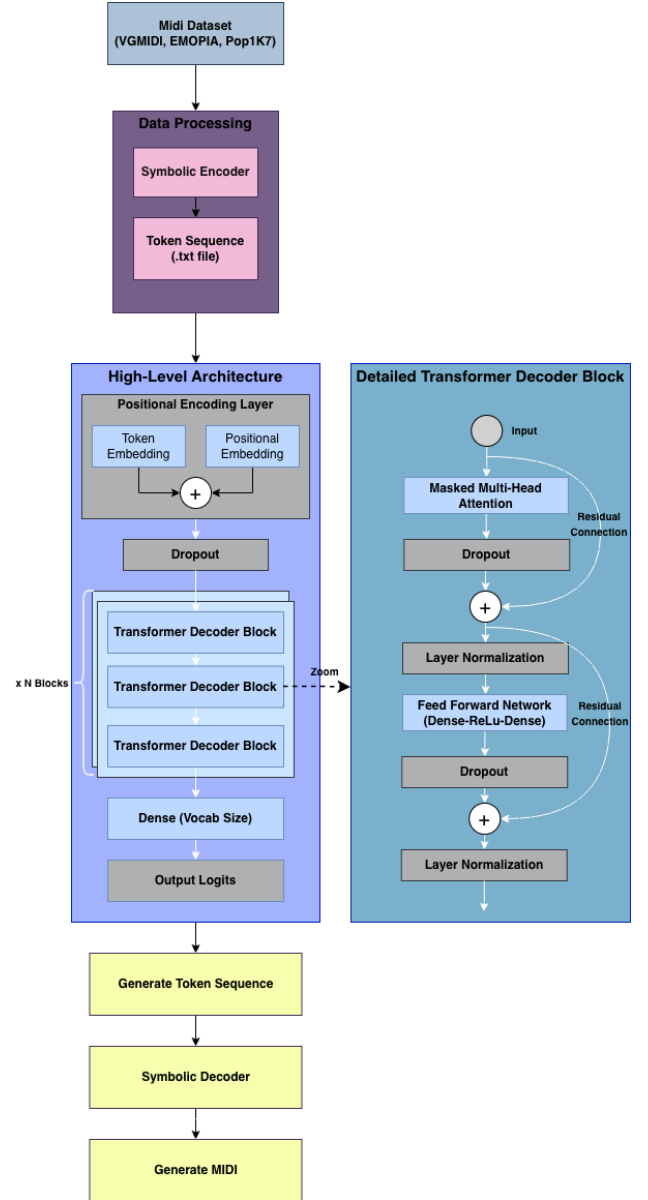


Figure 1. Overall pipeline of data processing, training and generating MIDI

done with the “onsets-and-frames” RNN-based piano transcription model [10]. This dataset is the largest among the three, contributing 1,577,835 tokens to the final data mix.

The VGMIDI dataset [3] is composed of 728 pieces extracted from video game soundtracks. All the pieces are piano arrangements of the soundtracks and vary in length from 26 seconds to 3 minutes. This dataset is the smallest among the three, contributing 202,030 tokens to the final data mix.

Consequently, the final combined dataset constitutes 2.25M tokens of transcribed piano music.

3.2 Model Architecture

We utilize a decoder-style Transformer. The input to the model at time t is a projected embedding of the music token. The architecture consists of 2 stacked decoder blocks, each employing a self-attention layer that computes:

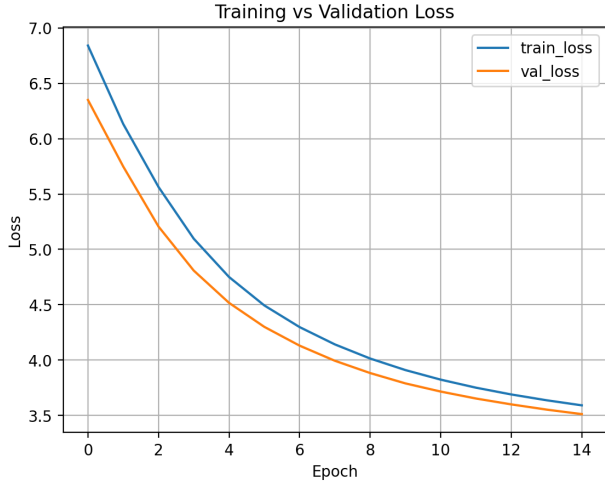


Figure 2. Training loss and validation Loss curve. Both loss and validation plateau after 15 epoch.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Where d_k is the embedding dimension which we set to 128. We use 8 attention heads and a Feed-Forward Neural Network size of 512 and a sequence length (attention window size) of 256 tokens. This proposed architecture yields a model with approximately 1.7M trainable parameters.

3.3 Training

The model was trained on the combined corpus using a cross-entropy loss function. We monitored training progress by inspecting generated sequences at various intervals to distinguish between chaotic "noise" and structured "music". We trained our model using the A100 GPU backend provided by Google Colab. Training was conducted using different hyperparameter configurations for the model architecture as well as for the training subset.

We tried training on smaller subsets of the data, as well as scaling the dataset up to more than 17M tokens by employing techniques like time stretching, making each piece up to 5% faster or slower and transposition where lowering or raising the pitch of each piece by up to a major third will produce another training sample.

4. RESULTS

The loss curve in Figure 2 shows that training loss decreased from 6.8 to 3.6 and validation loss also decreased from 6.4 to 3.5 after 15 epoch of training. This indicates that model successfully learned the features of dataset and model was neither overfitted nor underfitted.

During generation, we introduce a `-seqinit` parameter that constrains the model to begin each sequence with a predefined set of tokens. This mechanism provides the model with an explicit musical context at generation time and allows controlled similarity to a reference piece by modifying the initial token sequence.

n-gram level	Structural Cosine Similarity
2-gram	0.0286
3-gram	0.0185
4-gram	0.0011

Table 1. Structural cosine similarity between generated sequences and reference music before training.

n-gram level	Structural Cosine Similarity
2-gram	0.8994
3-gram	0.8065
4-gram	0.7128

Table 2. Structural cosine similarity between generated sequences and reference music after training.

Prior to training, the model produces near-random token distributions that lack musical coherence (e.g., `t_120 v_56 d_whole_0 n_40 w_1599`). After training, the generated output adheres to valid symbolic music syntax, consistently ordering tempo (t), velocity (v), duration (d), and note (n) tokens (e.g., `t_120 v_56 d_eighth_0 v_60 d_quarter_0 n_79`), indicating that the model has learned basic structural constraints of the representation.

To quantitatively evaluate structural consistency, we compute the Structural Cosine Similarity between generated sequences and a reference piece across multiple n -gram levels. As shown in Table 1, untrained generations exhibit negligible structural similarity at all n -gram orders, reflecting the absence of learned musical dependencies. In contrast, Table 2 demonstrates a substantial increase in similarity after training, with consistently high scores across 2-gram to 4-gram levels.

This trend suggests that the transformer effectively captures local and mid-range structural patterns in symbolic music, contributing to more coherent musical sequences.

To further validate, we analyzed the relative frequency of token types over a fixed window comparing generated against reference ground-truth sequences from one of the songs. The distributions of note (n), velocity (v), and duration (d) tokens were highly similar between generated and real data. For example, in the token range [50:150], the ground-truth sequence contained 38 note tokens, 31 velocity tokens, and 30 duration tokens, while the generated sequence contained 46, 29, and 25 respectively.

5. DISCUSSION AND ERROR ANALYSIS

While the Structural Cosine Similarity shows clear evidence that the model is learning, the quality of the generated music was extremely low. Since the metrics indicated that this issue did not originate from the training stage, we conducted an error analysis at an earlier data processing stage.

To rule out dataset corruption as a contributing factor, we trained the model on each dataset individually and on all possible combinations of the datasets (six configurations in total). None of these configurations yielded a

noticeable improvement in generation quality, suggesting that the observed failures do not originate from the training data.

To isolate the source of failure, we performed an encoder–decoder round-trip test on the training data. A reference MIDI file was first encoded into symbolic tokens and then decoded back into MIDI using our implementation based on [3]. If the encoder and decoder were correct, this process should approximately reconstruct the original piece.

However, the decoded output was consistently corrupted: the reconstructed MIDI files were dramatically shorter in duration and exhibited severe temporal distortion. This failure occurred even when no model inference was involved, indicating that the issue was independent of training or generation.

To verify that the dataset itself was not corrupted, we repeated the same round-trip experiment using `miditok` [11], a widely used MIDI tokenization library. In contrast, the `miditok`-based encoder and decoder successfully reconstructed the original music with correct timing and pitch structure.

This experiment conclusively demonstrates that the primary source of error lies in the custom encoder–decoder implementation rather than in the Transformer model.

6. CONCLUSION

This work investigated the possibility of transitioning from an LSTM architecture to Transformer-based models for symbolic music generation. It operated under the hypothesis that high-capacity models trained on aggregated datasets would resolve the structural "drift" often observed in LSTMs. By training a decoder-only Transformer with a dataset of 2.25 million tokens from VGMIDI, EMOPIA, and Pop1K7, we established a "Big Data" baseline.

Our quantitative results strongly suggest that the Transformer architecture successfully captured the complex dependencies of symbolic music. The convergence of training and validation loss and high n-gram Structural Cosine Similarity demonstrate that the model effectively learned the syntax and hierarchical structure. Furthermore, the distributions of note, velocity, and duration tokens in the generated sequences closely matched the ground truth, confirming the model's ability to replicate statistical musical patterns.

However, despite these promising metrics, the quality of the generated audio remained low. Our subsequent error analysis isolated the root cause of this discrepancy as a critical flaw in the symbolic encoder-decoder implementation, rather than the model architecture itself. Round-trip tests confirmed that while the model correctly predicted tokens, the decoding stage introduced severe temporal distortions and truncation.

In conclusion, this study confirms that the Transformer architecture maintains structural coherence. Future work should focus on integrating robust tokenization libraries like `miditok` to resolve the decoding issues, further scaling

the dataset, and re-introducing the emotion-based conditioning available in the EMOPIA dataset.

Disclosing the Use of AI

The authors utilized generative AI tools, specifically ChatGPT and Google Gemini, solely for the purposes of spell-checking and grammatical revision during the preparation of this manuscript. No AI tools were used for data generation, analysis, or the formulation of scientific conclusions.

7. REFERENCES

- [1] H. Davis and S. M. Mohammad, "Generating music from literature," *arXiv preprint arXiv:1403.2124*, 2014. [Online]. Available: <https://arxiv.org/abs/1403.2124>
- [2] W.-Y. Hsiao, J.-Y. Liu, Y.-C. Yeh, and Y.-H. Yang, "Compound word transformer: Learning to compose full-song music over dynamic directed hypergraphs," *arXiv preprint arXiv:2101.02402*, 2021. [Online]. Available: <https://arxiv.org/abs/2101.02402>
- [3] L. N. Ferreira and J. Whitehead, "Learning to generate music with sentiment," in *Proceedings of the 20th International Society for Music Information Retrieval Conference (ISMIR)*, 2019.
- [4] D. Herremans and E. Chew, "Morpheus: Generating structured music with constrained patterns and tension," *arXiv preprint arXiv:1812.04832*, 2018. [Online]. Available: <https://arxiv.org/abs/1812.04832>
- [5] M. Scirea, J. Togelius, P. Eklund, and S. Risi, "Affective evolutionary music composition with metacompose," *Genetic Programming and Evolvable Machines*, vol. 18, no. 1, pp. 1–33, 2017.
- [6] S. Ji, J. Luo, and Y.-H. Yang, "A survey on deep learning for symbolic music generation: Representations, algorithms, evaluations, and challenges," *ACM Computing Surveys*, vol. 56, 2023.
- [7] H.-T. Hung, J. Ching, S. Doh, N. Kim, J. Nam, and Y.-H. Yang, "Emopia: A multi-modal pop piano dataset for emotion recognition and emotion-based music generation," *arXiv preprint arXiv:2108.01374*, 2021.
- [8] S. Oore, I. Simon, S. Dieleman, and D. Eck, "Learning to create piano performances," in *Proceedings of the NIPS Workshop on Machine Learning for Creativity*, Long Beach, CA, USA, 2017.
- [9] Y.-S. Huang and Y.-H. Yang, "Pop music transformer: Beat-based modeling and generation of expressive pop piano compositions," in *Proceedings of the 28th ACM International Conference on Multimedia (MM '20)*, New York, NY, USA, 2020, pp. 1180–1188.
- [10] C. Hawthorne, E. Elsen, J. Song, A. Roberts, I. Simon, C. Raffel, J. Engel, S. Oore, and D. Eck, "Onsets and frames: Dual-objective piano transcription," *arXiv*

preprint arXiv:1710.11153, 2017. [Online]. Available:
<https://arxiv.org/abs/1710.11153>

- [11] N. Fradet, J.-P. Briot, F. Chhel, A. E. F. Seghrouchni, and N. Gutowski, “miditok: A python package for midi file tokenization,” 2023. [Online]. Available:
<https://arxiv.org/abs/2310.17202>